Date __/__/____
Intimate

— SLIDING WINDOW —

=> Pattern -

01. Fixed-size Sliding Window - You need to compute something ~~all~~ over all subarrays/substrings of a fixed size k.
Leetcode 643 - Maximum Average Subarray I

02. Variable-size Sliding window - You're finding the shortest/longest subarray/substring that satisfies a condition.
Leetcode 209 - Minimum Size Subarray Sum
Leetcode 3 - Longest Substring without Repeating Characters.
When you need to expand/shrink the window based on a condition.

03. Character Frequency Tracking (String-specific.) - You need to track frequency/count of characters or elements inside the window.

Leetcode 567 - Permutation in String
Leetcode 438 - Find All Anagrams in a String

Date / /
Intimate

01. Count occurrences of Anagrams
02. Find All Anagrams in a String
03. Minimum Size Subarray Sum
04. First negative integer in every window of size k
05. Minimum window Substring
06. Contains Duplicate - II
07. Count Subarrays with Fixed Bounds
08. Maximum number of Vowels in a Substring of Given length
09. K Radius Subarray Averages
10. Longest Subarray of 1's After Deleting One Element
11. Maximize the Confusion of an Exam
12. Sliding window maximum
13. Frequency of the most frequent Element
14. Count Complete Substrings
15. Binary Subarrays with Sum
16. Subarray Product less than K.
17. Length of Longest Subarray with at most K- frequency
18. Count Subarrays where max Element appears at least k
19. Subarrays with k different integers
20. Get Equal Substring within Budget
21. Find Subarray with Bitwise AND Closest to K.
22. Grumpy Bookstore Owner
23. Longest Continuous Subarray with Absolute diff less than
24. Minimum Swaps to Group All 1's together II
25. Count substring that satisfy K-constraint I & II
26. Permutation in String
27. Find the power of K-size Subarrays I
28. ~~Find~~ Shortest Subarray with Sum at Least K
29. Maximum Sum of distinct Subarrays with length K.
30. Take K of Each Character from left and Right
31. Continuous Subarrays
32. Alternating Groups II
33. Count of Substrings Containing Every Vowel and

Date __/__/____
Intimate

34. Number of Substrings Containing All three characters
35. Longest Nice Subarray
36. Count the no. of good subarrays
37. Count complete subarrays in an Array
38. Count Subarrays with score less than K.
39. Maximum difference between Even and odd frequency II
40. Reschedule Meetings for Maximum Free Time I
41. Maximum Erasure Value
42. Fruit Into Baskets
43.

Date __/__/____
Intimate

01. Count Occurrences of Anagrams –

Input :- txt = forxxorfxdofr

pat = for

Output :- 3

Anagram of for
⇓
for
foo
ofr
orf
rof
rfo

(1) i & j
(2) Shift
(3) Frequency Update

BRUTE FORCE –

f	o	r	x	x	o	r	f	x	d	o	f	r
0	1	2	3	4	5	6	7	8	9	10	11	12

f	o	r
1	1	1

(1) Start with 0th index go till ~~3rd~~ 2nd index and check if for ass's all index has become 0 if yes cnt++

(2) move to index 1st to 3rd → follow same procedure.

TC – O(M * N)

Date / / Intimate

//Code:-

```
private boolean areAnagram (String s1, String s2) {
    int [] cnt = new int [26];

    for (int i = 0; i < s1.length(); i++) {
        cnt [s1.charAt(i) - 'a' ]++;
    }

    for (int i = 0; i < s2.length(); i++) {
        cnt [s2.charAt(i) - 'a'] --;
    }

    for (int i = 0; i < 26; i++) {
        if (cnt [i] != 0) {
            return false;
        }
    }

    return true;
}
```

TC- O(N * M)
SC- O(1)

```
public int search (String pat, String txt) {
    int res = 0;
    int n = txt.length();
    int m = pat.length();

    for (int i = 0; i <= n-m; i++) {
        if (areAnagram (txt.substring (i, i+m), pat)) {
            res++;
        }
    }

    return res;
}
```

Date __/__/__ Intimate

⇒ Optimal ⇒ Sliding Window -

txt =

f	o	r	x	x	o	r	f	x	d	o	f	r
0	1	2	3	4	5	6	7	8	9	10	11	12

i j

pat =

f	o	r
0	1	2

DRY RUN :-

f	o	r
1	1	1

(1)

f	o	r	x	x	o	r	f	x	d	o	f	r
0	1	2	3	4	5	6	7	8	9	10	11	12

i j j

cnt = 0

j - i + 1 == pat.length()

0	0	0	⇒ arr
f	o	r	

& arr's all elements count is zero.

cnt = 1

i++

1	0	0
f	o	r

(2)

f	o	r	x	x	o	r	f	x	d	o	f	r

i j

j - i + 1 == pat.length()

1	0	0
f	o	r

& arr's all elements count is not zero.

i++

1	1	0
f	o	r

(3)

f	o	r	x	x	o	r	f	x	d	o	f	r

i j

j - i == pat.length()

1	1	0
f	o	r

not zero

i++

1	1	1
f	o	r

① i & j
② Shift
③ frequency update

Date __/__/__
Intimate

(4)

f	o	r	x	x	o	r	f	x	d	o	f	r

i i j j

(i) j-i+1 == pat.length ?

f o r

1	0	1

an

not zero

i++

j++

(ii) j-i+1 == pat.length ?

1	0	1

f o r

not zero i++

j++

(5)

f	o	r	x	x	o	r	f	x	d	o	f	r

i j

j-i+1 == pat.length;

0	0	0

f o r

zero cnt = 2

i++;

0	1	0

f o r

(6)

f	o	r	x	x	o	r	f	x	d	o	f	r

i j

j-i+1 == pat.length;

0	1	0

f o r

not zero i++

j++

0	1	1

f o r

(7)

f	o	r	x	x	o	r	f	x	d	o	f	r

i j

j-i+1 == pat.length

0	1	1

f o r

not zero i++ j++

and so on...

```
public int search(String pat, String txt) {
    int n = txt.length();
    int[] counter = new int[26];

    // Count the frequency of each char in pattern
    for (int i=0; i<pat.length(); i++) {
        char ch = pat.charAt(i);
        counter[ch - 'a']++;
    }

    int i=0, j=0;
    int result = 0;
    int k = pat.length();

    ~~int[] tempCounter = counter.clone();~~
    while (j<n) {
        counter[txt.charAt(j) - 'a']--;
        // Decrease frequency of current character.
        if (j-i+1 == k) {
            if (allZero(counter)) {
                result++;
            }
            // Before Sliding window forward,
            restore count of character going
            out
            counter[txt.charAt(i) - 'a']++;
            i++;
        }
        j++;
    }
    return result;
}
```

Date __/__/__
Intimate

```
private boolean allZero(int[] arr){
    for(int val : arr){
        if(val != 0){
            return false;
        }
    }
    return true;
}
```

TC - O(m+N)	≈ O(2N)
SC - O(1)	

Every time window size reaches n,

you call allZero(count) → O(26) → O(1)

Constant.

Date __/__/__
Intimate

02 Find All Anagrams in a String -

Input :- txt = f o r x x o r f x d o f r
(0 1 2 3 4 5 6 7 8 9 10 11 12)

pat = for

Output :- {0, 5, 10} ⇒ Starting index

Anagrams
for
fro
ofr
orf
rof
rfo

Same as previous question with ~~s~~ a few changes in code -

① | f | o | r | x | x | o | r | f | x | d | o | f | r |
i
~~j~~ j j

j - i ~~+1~~ == pat.length;

cnt ++

i++ j++

| 0 | 0 | 0 |
f o r

② | f | o | r | x | x | o | r | f | x | d | o | f | r |
i j

j - i + 1 == pat.length;

| 1 | 0 | 0 |
f o r

not zero

i++ j++

//Code -

Date / /
Intimate

```
public List<Integer> findAnagram(String s, String p){
    int n = s.length();

    int freq[] = new int[26];
    for(char str : p.toCharArray()){
        freq[str - 'a']++;
    }

    int i = 0;
    int j = 0;

    List<Integer> ans = new ArrayList<>();

    while(j<n){
        freq[s.charAt(j) - 'a']--;

        if(j-i+1 == p.length()){
            if(CallZero(freq)){
                ans.add(i);
            }

            freq[s.charAt(i) - 'a']++;
            i++;
        }
        j++;
    }
    return ans;
}
```

Date __/__/__
Intimate

```
private boolean allZero(int[] arr){
    for(int val : arr){
        if(val != 0){
            return false;
        }
    }
    return true;
}
```

TC - O(M+N) ≈ O(2M)

SC - O(1)

Date __/__/__
Intimate

03. Minimum Size Subarray Sum –

Input:- target = 7 , nums = { 2, 3, 1, 2, 4, 3 }

Output – 2 (sum >= target)

DRY RUN – {Sliding Window}

(1)

0	1	2	3	4	5
2	3	1	2	4	3

i j

target = 7

MinLen = n

Sum = 2 < 7 j++

Sum = 5 < 7 j++

Sum = 6 < 7 j++

Sum = 8 >= 7 ✓ Length = 3-0 ⇒ 3+1 ⇒ 4

(2)

0	1	2	3	4	5
2	3	1	2	4	3

i i j

Sum = 6 < 7 j++ 4-1 ⇒ 3+1

Sum = 10 >= 7 Length = min(4, 4)

⇒ 4

(3)

0	1	2	3	4	5
2	3	1	2	4	3

i j

Sum = 7 >= 7 Length = min(4, 3)

⇒ 3

Sum = 6 < 7

(4)

0	1	2	3	4	5
2	3	1	2	4	3

i j

Sum ⇒ 9 { 7

7 { 3

Ans:

length ⇒ min(3, 3)

⇒ 3 ⇒ min(3, 2)

Date __/__/____
Intimate

Code Sliding Window —

```
public int minSubArraySum(int target, int nums[]){

    int n = nums.length;
    int minLength = Integer.MAX_VALUE;

    int i=0;
    int j=0;
    int sum=0;

    while(j<n){
        sum += nums[j];

        while(sum >= target){
            minLength = Math.min(minLength, j-i+1);

            sum -= nums[i];
            i++;
        }
        j++;
    }

    return minLength == Integer.MAX_VALUE ? 0 :
                                    minLength;

}
```

TC - O(2N)
SC - O(1)

Date / / Intimate

04. First Negative Integer in every window of size K -

Input = {-8, 2, 3, -6, 10}, K = 2

Output :- {-8, 0, -6, -6}

Brute force —

O(N * K)

(1) -8 2 3 -6 10
i
j

firstNeg = -8 ans {-8}

(2) -8 2 3 -6 10
i j

j-i<1

(3) -8 2 3 -6 10
i j

j-i+1 == k

firstNeg = 0

ans = {-8, 0}

(4) -8 2 3 -6 10
i
j

firstNeg = 0

(5) -8 2 3 -6 10
i j

firstNeg = -6

ans = {-8, 0, -6 ...}

Date __/__/__
Intimate

// Brute force -

```
public static List<Integer> firstNegInt(int arr[],
                                        int k){
    int n = arr.length;

    List<Integer> ans = new ArrayList<>();

    for(int i=0; i<=n-k; i++){

        for(int j=i; j<i+k; j++){

            if(arr[j]<0){

                firstNeg = arr[j];
                break;
            }
        }

        ans.add(firstNeg);
    }

    return ans;
}
```

Date __/__/__
Intimate

Optimal (Sliding Window) —

8	1	-2	2	-3	6	8	-1
0	1	2	3	4	5	6	7

K = 3

(1) i, j = 0

(i) deq = [8]

(ii) j = = 1

deq = [8, 1]

(iii) j = = 2

deq = [8, 1, -2]

j - i + 1 = = K

2 - 0 + 1 = = 3

ans = [-2]

i = = 1

deq = [-2]

(2)

8	1	-2	2	-3	6	8	-1
0	1	2	3	4	5	6	7

K = 3

i = 1, j = 3

j - i + 1 = = K

ans = [-2, -2]

deq = [-2, -2]

i = = 2

Date __/__/__
Intimate

(3) | 8 | 1 | -2 | 2 | -3 | 6 | 8 | -1 | K = 3
0 1 2 3 4 5 6 7
i j

deq = [-2 | -3]

j - i + 1 == K

ans = [-2 | -2 | -2]

deq = [-3]

i == 3

(4) | 8 | 1 | -2 | 2 | -3 | 6 | 8 | -1 | K = 3
0 1 2 3 4 5 6 7
i j

deq = [-3]

j - i + 1 == K

ans = [-2 | -2 | -2 | -3]

deq = [-3]

i == 4

(5) | 8 | 1 | -2 | 2 | -3 | 6 | 8 | -1 | K = 3
i j

deq = [-3]

j - i + 1 == K

ans = [-2 | -2 | -2 | -3 | -3]

deq = [ ]

i == 5

Date __/__/__
Intimate

//Code -

```
public List<Integer> firstNegativeInWindow(int[] A,
        int K){
    int i = 0, j = 0;
    int N = A.length;
    List<Integer> result = new ArrayList<>();
    Deque<Integer> deque = new LinkedList<>();

    while(j < N){
        if(A[j] < 0){
            deque.addLast(A[j]);
        }

        if(j-i+1 == K){

            if(!deque.isEmpty()){
                result.add(deque.peekFirst());
            }
            else{
                result.add(0);
            }
        //Before Sliding window, remove element
            if(!deque.isEmpty() &&
                    A[i] == deque.peekFirst()){
                deque.pollFirst();
            }
            i++; // Print window
        }
        j++; // expand window
    }
    return result;
}
```

Date __/__/____
Intimate

05. Minimum Window Substring –

Input :- S = | A | D | O | B | E | C | O | D | E | B | A | N | C |

t = "ABC"

Output :- 4

Dry Run :-

	0	1	2	3	4	5	6	7	8	9	10	11	12
S =	A	D	O	B	E	C	O	D	E	B	A	N	C
	i, j												

t = ABC

countRequired = 3

map

① S = | A | D | O | B | E | C | O | D | E | B | A | N | C |

i, j

countRequired = 2

A	1 → 0
B	1
C	1
D	

② S = | A | D | O | B | E | C | O | D | E | B | A | N | C |

i, j

countReq = 1

map

A	0
B	1 → 0
C	1

Intimate

③ S = | A | D | O | B | E | C | O | D | E | B | A | N | C |
i j

map

reqCnt = 0

A	1
B	0
C	0

cntReq == 0

minWindowSize = j - i + 1 (Current Size)

Start_i = i;

map =>

A	1
B	0
C	0

j++

cntReq = 1

④ S = | A | D | O | B | E | C | O | D | E | B | A | N | C |
i j

A	1
B	0
C	0

⑤ S = | A | D | O | B | E | C | O | D | E | B | A | N | C |
i j

A	1	0
B	-1	
C	0	

Date / /
Intimate

//Code:-

```
public String minWindow ( String s, String t) {
    int n = s.length();

    if ( t.length() > n) {
        return "";
    }

    Map < Character, Integer > mp = new HashMap <> ();

    // Build frequency map for characters in t
    for ( char ch : t.toCharArray()) {
        mp.put (ch, mp.getOrDefault (ch, 0) + 1);
    }

    int requiredCount = t.length();
    int i = 0, j = 0;
    int minWindowSize = Integer.MAX_VALUE;
    int startI = 0;

    while (j < n) {
        char ch = s.charAt(j);

        if (mp.containsKey (ch)) {
            if (mp.get(ch) > 0 ) {
                requiredCount--;
            }
            mp.put (ch, mp.get(ch) - 1);
        }
```

Date __/__/__
Intimate

```
// Start shrinking the window if all characters are matched

while (requiredCount == 0) {
    int currWindowSize = j - i + 1;

    if (currWindowSize < minWindowSize) {
        minWindowSize = currWindowSize;
        startI = i;
    }

    char leftChar = s.charAt(i);   → अई

    if (mp.containsKey(leftChar)) {
        mp.put(leftChar, mp.get(leftChar) + 1);

        if (mp.get(leftChar) > 0) {
            requiredCount++;
        }
    }
    i++;
}
j++;
}
return minWindowSize == Integer.MAX_VALUE ?
        "" : s.substring(startI, startI + minWindowSize);
}
```

Date / / Intimate

Q: Contains Duplicate II -

Input :- [1, 2, 3, 1] k = 3

Output :- true.

abs(0-3)
3 <= 3

DRY RUN :-

(1) (j - i) <= k

[1, 2, 3, 1]
i
j

DRY RUN

(1) [1, 2, 3, 1]
i
j

set [1]

(2) [1, 2, 3, 1]
i j j j

j - i = 2 - 0 <= k
j - i = 3 <= k

set = [1 | 2] j - i <= k
set = [1 | 2 | 3]

set = [1 | 2 | 3 | 1]

| 1 | 2 | 3 | 1 | 2 | 3 |

False Ans.

(3) [1, 2, 3, 1]
i j

set. contains (j) ✓

return true ✓

// Code (Sliding window) -

Date __/__/__
Intimate

```
public boolean containsNearbyDuplicate(int[] nums,
        int k){

    Set<Integer> set = new HashSet<>();
    int i=0, j=0;
    int n = nums.length;

    while(j<n){
        //Shrink the window if it exceeds size k
        if(j-i > k){
            set.remove(nums[i]);
            i++;
        }
        // if duplicates found within range
        if( set.contains(nums[j])){
            return true;
        }
        set.add(nums[j]);
        j++;
    }

    return false;
}
```

Microsoft # Meesho

Date __/__/__ Intimate

07. Count Subarray with Fixed Bounds -

Input - nums = {1, 3, 5, 2, 7, 5}
minK = 1, maxK = 5

Output - 2 (दो subarray मिले है जिनका minValue = 1 & maxValue = 5 है।)

DRY RUN -

(1)

6	3	5	2	1
0	1	2	3	4

minK = 1
maxK = 5

ans = 2 ({5, 2, 1}, {3, 5, 2, 1})

(2)

1	3	5	2	7	5
0	1	2	3	4	5

idx ⇒ 0 1 2 3 4 5

ans =
minK = 1 maxK = 5

minKPosition = -1
maxKPosition = -1
culpritIdx = -1

Smaller = min (0, 2) = 0

temp = 0 - (-1) = 1

ans +=

Date / / Intimate

① | 1 | 3 | 5 | 2 | 7 | 5 |
  ↑ i

minkPosition = 0 maxkPosition = -1
culpritIdx = -1

smaller = min (0, -1) = -1

temp = -1 - (-1) ⇒ -1 + 1 = 0

ans += temp <= 0 ? 0 : temp

ans ⇒ 0

② | 1 | 3 | 5 | 2 | 7 | 5 |
  ↑ i ↑ i

smaller = min(0, 2) = 0
temp = 0 - (-1) ⇒ 1

minkPosition = 0
maxkPosition = 2
culpritIdx = -1

ans += temp ⇒ ans = 1

③ | 1 | 3 | 5 | 2 | 7 | 5 |
  ↑ i

minkPosition = 0 → 2
maxkPosition = 2
culpritIdx = -1

smaller = min (0, 2) = 0
temp = 0 - (-1) = 1

ans += temp

ans = 2

Date __/__/__
Intimate

(4) | 1 | 3 | 5 | 2 | 7 | 5 |
4

minKPosition = 0
maxKPosition = 2
culpritIdx = 4

Smaller = min (0, 2) = 0
temp = 0 − (4) ⇒ − 4

ans = − 4

(5) | 1 | 3 | 5 | 2 | 7 | 5 |
4

minKPosition = 0
maxKPosition = 5
culpritIdx = 4

smaller = min (0, 5) = 0

temp = 0 − 4 = − 4

ans += 0

// Code -

```
public long countSubarrays(int[] nums,
    int minK, int maxK) {

    long ans = 0;

    int minKPosition = -1;
    int maxKPosition = -1;
    int culpritIdx = -1;

    for(int i=0; i<nums.length; i++){
        if(nums[i] < minK || nums[i] > maxK){
            culpritIdx = i; // Invalid no.
        }

        if( nums[i] == minK){
            minKPosition = i;
        }

        if(nums[i] == maxK ){
            maxKPosition = i;
        }

        int Smaller = Math.min(minKPosition,
                               maxKPosition);
        long temp = Smaller - culpritIdx;

        ans += (temp <= 0)?0 : temp;
    }
    return ans;
}
```

microsoft

Date __/__/__
Intimate

08. Maximum Number of Vowels in a Substring of Given Length –

Input:- S = "abciiidef", K = 3

Output:- 3

DRY RUN:–

① "a b c o o o d e f" K=3
0 1 2 3 4 5 6 7 8
i
j

⇒ count = 0

⇒ maxV = 0

② "a b c o o o d e f"
i j

vowel = 1

③ "a b c o o o d e f"
i j

vowel = = 1

j-i+1 == K

maxV = 1

a[i] = vowel = (a)

vowel = 0

i ++

④ "a b c o o o d e f"
i j

vowel = 1

Date / / Intimate

(5) a b c o o o d e f
i j
vowel == 2
maxVowel == 2

(6) a b c o o o d e f
i j
vowel = 3
maxVowel == 3

vowel == 2 i ++

(7) a b c o o o d e f
i j
vowel == 1 vowel = 2
i ++

(8) a b c o o o d e f
i j
vowel == 1 vowel = ~~2~~
i ++

→ window: two variables i=0, j=0
→ keep counting vowel if s[j] is 'a'||'e'||'i'||'o'||'u'
→ if <-- k --> (j - i + 1 == k) {
i j
- You found a window of size k
- Update maxV
- Also shift window right side. (i++)
- check if s[i] was vowel or not

Date __/__/__
Intimate

//Code:-

```
public int maxVowels(String s, int k){
    int n = s.length();
    int i = 0, j = 0;
    int maxV = 0;
    int count = 0;

    while(j<n){
        if(isVowel(s.charAt(j)){
            count++;
        }

        if(j-i+1 == k){
            maxV = Math.max(maxV, count);

            if(isVowel(s.charAt(i))){
                count--;
            }
            i++;
        }
        j++;
    }
    return maxV;
}
```

Date __/__/__
Intimate

09. K-Radius Subarray Averages - {Amazon}

Input:- nums - [7, 4, 3, 9, 1, 8, 5, 2, 6]

D

(2) Input:- nums = | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

n = 7
K = 2

Output:- result = | -1 | -1 | 3 | 4 | 5 | -1 | -1 |

Brute force :-

```
for (i=0; i<n; i++) {
    for (left → k) {
    }
    for (right side) {
    }
}
```

Date __/__/__
Intimate

Better1 (Prefix Sum) –

Dry Run :–

nums = | 1 | 2 | 3 | 4 | 1 | 6 | 7 |

k = 2, n = 7

i = (n - k) - 1
⇒ 5 - 1 ⇒ 4

prefixSum = | 1 | 3 | 6 | 10 | 11 | 17 | 24 |

Result – | -1 | -1 | ✗ | ✗ | ✗ | -1 | -1 |

2 3 4

i = i + k → (n - k) - 1

(1) | 1 | 2 | 3 | 4 | 1 | 6 | 7 |

11/5 ⇒ 2
↳ element

(2) | 1 | 2 | 3 | 4 | 1 | 6 | 7 |

– minus

17 - 1 ⇒ 16/5 ⇒ 3

(3) | 1 | 2 | 3 | 4 | 1 | 6 | 7 |

24 - 3 ⇒ 21/5 ⇒ 4

Result = | -1 | -1 | 2 | 3 | 4 | -1 | -1 |

Date __/__/__
Intimate

⇒ Story Points -

(•) Base → (k == 0) ~ nums return

(•) Prefix Array (-1) = {-1, -1, -1}

(•) k ü k = (2 * k + 1) üf (n < 2 * k + 1){
Prefix Array

(•) for (ü = k; ü <= n-k-1; ü++){
int leftida = ü - k;
int rightida = ü + k;

Sum = prefix (i+k) ~~prefix [i-k-1]~~

üf (i - k > 0) {
} Sum -= prefix [i-k-1]

avg = Sum / (2*k+1);

result [i] = avg

return result;

Date __/__/__
Intimate

// Code - (Prefix Sum) -

```
public int[] getAverages(int[] nums, int k){
    int n = nums.length;

    if(k==0){
        return nums;
    }

    int result[] = new int[n];
    Arrays.fill(result, -1);

    if(n < 2*k+1){
        return result;
    }

    long[] prefixSum = new long[n];
    prefixSum[0] = nums[0];

    for(int i=1; i<n; i++){
        prefixSum[i] = prefixSum[i-1] + nums[i];
    }

    for(int i=k; i<=n-k-1; i++){
        int leftIdx = i-k;
        int rightIdx = i+k;
        long sum = prefixSum[rightIdx];
        if(leftIdx > 0){
            sum -= prefixSum[leftIdx-1];
        }
        result[i] = (int)(sum/(2*k+1));
    }

    return result;
}
```

// Solution Using (Sliding window) —

(1) left i right

nums = | 1 | 2 | 1 | 1 | 3 | 1 | 2 | 1 | 1 |
0 1 2 3 4 5 6 7 8

K = 3

11/7 => 1

result = | -1 | -1 | -1 | 1 | -1 | -1 | -1 | -1 | -1 |
0 1 2 3 4 5 6 7 8

(2) left i right

| 1 | 2 | 1 | 1 | 3 | 1 | 2 | 1 | 1 |

11 - 1 + 1 => 11/7 => 1

result = | -1 | -1 | -1 | 1 | -1 | -1 | -1 | -1 | -1 |
0 1 2 3 4 5 6 7 8
1

(3) left i right

| 1 | 2 | 1 | 1 | 3 | 1 | 2 | 1 | 1 |

1 11 - 2 + 1 => 10/7 => 1

result = | -1 | -1 | -1 | 1 | 1 | -1 | -1 | -1 | -1 |
0 1 2 3 4 5 6 7 8

result => | -1 | -1 | -1 | 1 | 1 | 1 | -1 | -1 | -1 |
0 1 2 3 4 5 6 7 8

(•) l = 0 , r = 2*K i = K

(•) window = findSum

avg = (window) / (2*K+1)

result[i] = avg;

i++

right++;

while (right < n)

(•) while (right < n) {
new = nums[right];
gone = nums[left];
result[i] = (window + new - gone) / (2*k+1);
i++
right++
left++
}
return result;

// Code (Sliding Window) —

```
public int[] getAverages(int nums[], int k) {
    int n = nums.length;

    if (k == 0) {
        return nums;
    }

    int result[] = new int[n];
    Arrays.fill(result, -1);

    if (n < 2*k+1) {
        return result;
    }

    long window = 0;
    int left = 0;
    int right = 2*k;
    int i = k;
```

Date __/__/__
Intimate

```
// Build the initial window sum
for (int j = left; j <= right; j++) {
    window += nums[j];
}

int count = 2 * k + 1;
result[i] = (int) (window / count);

i++;
right++;

// Slide the window
while (right < n) {
    window += nums[right];
    window -= (int) nums[left];

    result[i] = (int) (window / count);

    left++;
    right++;
    i++;
}
return result;
}
```

Date __/__/__
Intimate

10. Longest Subarray of 1's After Deleting One Element -

Example:- ① nums = {1, 1, 0, 1} → {1, 1, 1}
output = 3

② nums = {0, 1, 1, 1, 0, 1, 1, 0, 1} → {0, 1, 1, 1, 1, 1, 0, 1}
output = 5

③ nums = {1, 1, 1} → {1, 1}
output = 2

Brute force :-

```
public int longestSubarray(int nums[]) {
    int n = nums.length;
    int result = 0;
    int countZero = 0;

    for (int i = 0; i<n; i++) {
        if (nums[i] == 0) {
            countZero++;
            result = Math.max(result, findMax(nums, i));
        }
    }
    if (countZero == 0) {
        return n-1;
    }
    return result;
}
```

```
private int findMax(int nums[], int skipIndex) {
    int maxLen = 0, currLen = 0;

    for (int i = 0; i < nums.length; i++) {
        if (i == skipIndex) continue;

        if (nums[i] == 1) {
            currLen++;
            maxLen = Math.max(maxLen, currLen);
        } else {
            currLen = 0;
        }
    }

    return maxLen;
}
```

Approach (Using Sliding Window) –

① nums =

1	1	0	0	1	1	1	0	1
0	1	2	3	4	5	6	7	8

i, j

i = 0 j = 0

② zeroCount = ✗ ✗ 1

result = 0 ✗ ✗ ✗ 4

1	1	0	0	1	1	1	0	1

i j

ans = 1 - 0 → 1

③

0	1	2	3	4	5	6	7	8
1	1	0	0	1	1	1	0	1

i j

cnt = 1

ans = 2 - 0 = 2

④

1	1	0	0	1	1	1	0	1

i j

cnt = 2

nums[j] == 1

cnt = 2 ✓

i ✓ p

ans = 2

⑤

1	1	0	0	1	1	1	0	1

i ~~i~~ i j

cnt == 2

nums[i] == 1

cnt == 2

i ✓ ✓

cnt == 2

nums[i] = 0

cnt = 1

i ✓ ✓

// Code Sliding Window (Khandaani Approach)

Date __/__/__ Intimate

```
public int longestSubarray(int[] nums){
    int zeroCount = 0;
    int maxLength = 0;
    int i = 0;

    for (int j = 0; j < nums.length; j++){
        if (nums[j] == 0){
            zeroCount++;
        }

        while (zeroCount > 1){
            if (nums[i] == 0){
                zeroCount--;
            }
            i++;
        }
        maxLength = Math.max(maxLength, j-i);
    }
    return maxLength;
}
```

{1, 0, 0, 0, 0, 0}

Ex.

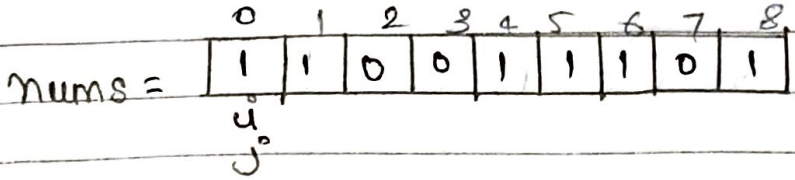
T.C - O(N)

S.C - O(1)

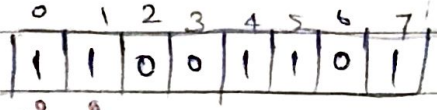
we must delete one element

Date __/__/__
Intimate

// Clean Code for Sliding Window – DRY RUN –



i = 0, j = 0

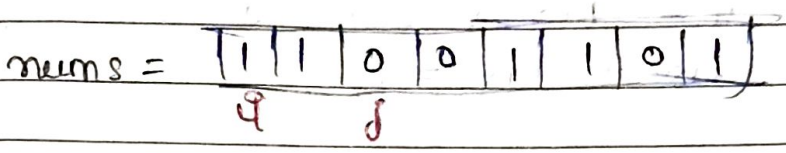
① 

last_zero_idx = ~~-1 2 3~~

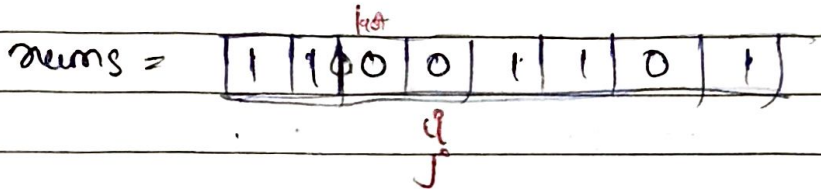
result = ~~0 1 2 3~~ 4

ans = 1

i = last_zero_idx + 1

② 

last Idx + 1 ⇒ i

③ 

// Clean Code —

```
public int longestSubarray(int[] nums){
    int i = 0;
    int j = 0;
    int lastZeroIdx = -1;
    int result = 0;

    while(j < nums.length){
        if(nums[j] == 0){
            i = lastZeroIdx + 1;
            lastZeroIdx = j;
        }
        result = Math.max(result, j-i);
        j++;
    }

    return result;
}
```

TC —

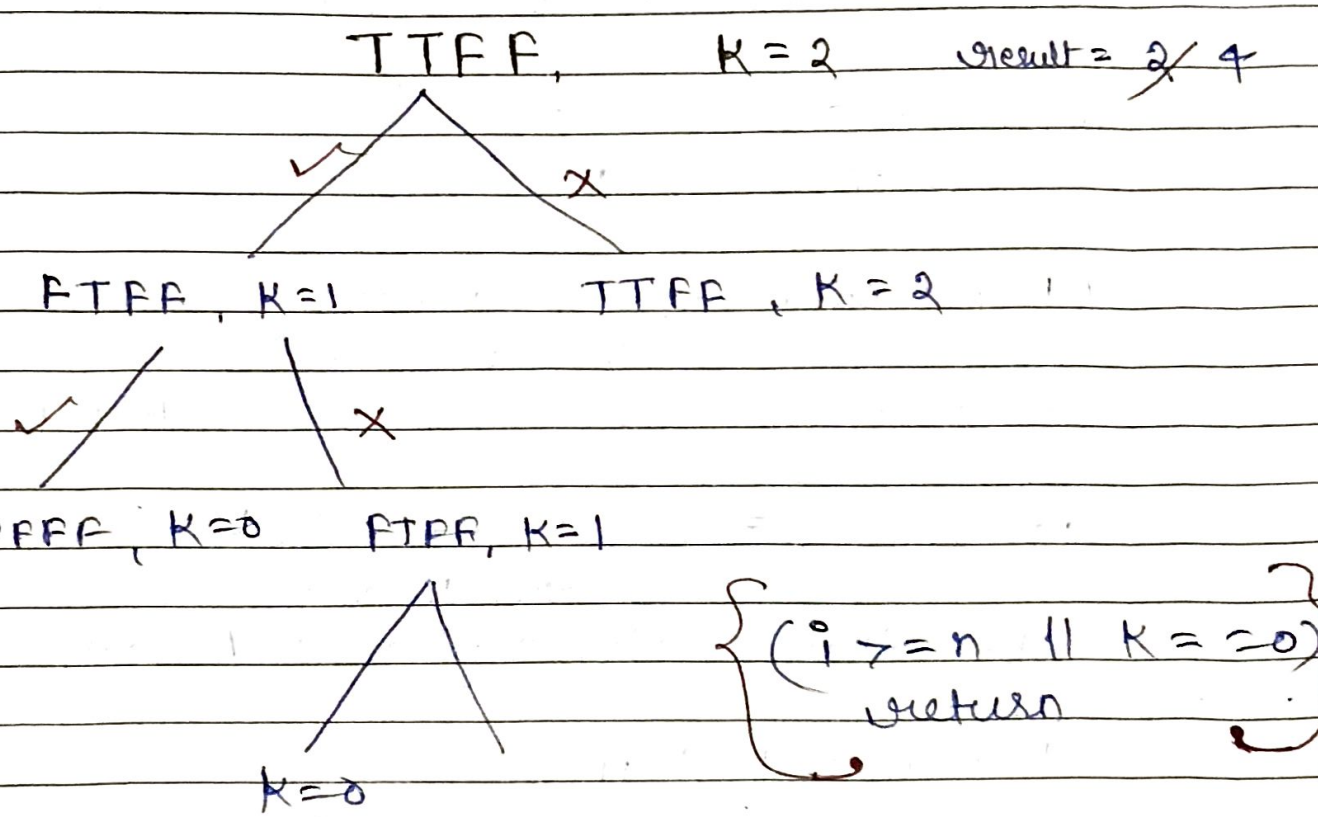
Date __/__/__
Intimate

11. Maximize the Confusion of an Exam -

Ex. (1) answerKey = "TTFF" , K = 2
Output - 4

(2) answerKey = "TFFT", K = 1
Output = 3

— BRUTE FORCE —



{ (i >= n || K == 0) return }

TC - O(2^n * n)

Date __/__/__
Intimate

// Brute force Code -

```
public int maxConsecutiveAnswers (String answerKey,
                                  int k) {
    return solve(0, new StringBuilder(answerKey),
                 k, answerKey.length(), 0);
}

private int solve(int idx, StringBuilder sb,
                  int k, int n, int currentMax) {
    int localMax = findMax(sb.toString());
    currentMax = Math.max(currentMax, localMax);

    if (idx >= n || k <= 0) {
        return currentMax;
    }
    // Try flipping current character
    char original = sb.charAt(idx);
    sb.setCharAt(idx, original == 'T' ? 'F' : 'T');

    int flipUsed = solve(idx+1, sb, k-1, n,
                         currentMax);
    sb.setCharAt(idx, original); // backtrack

    Try without flipping
    int noFlip = solve(idx+1, sb, k, n, currentMax);

    return Math.max(flipUsed, noFlip);
}
```

Date __/__/__
Intimate

```
private int findMax (String s) {
    int maxLen = 0;
    int count = 1;

    for (int i = 1; i < s.length(); i++) {
        if (s.charAt(i) == s.charAt(i-1)) {
            count++;
        }
        else {
            count = 1;
        }
        maxLen = Math.max(maxLen, count);
    }

    return Math.max(maxLen, 1);
}
```

Two Pass -

// Code (Sliding Window) -

```
public int maxConsecutiveAnswers (String
        answerKey , int k) {
    int n = answerKey.length();
    int result = 0;
    // first pass:
    int i = 0;
    int j = 0;
    int countF = 0;

    while (j < n) {
        if (answerKey.charAt (j) == 'F') {
            countF++;
        }

        while (countF > k) {
            if (answerKey.charAt(i) == 'F') {
                countF--;
            }
            i++;
        }

        result = Math.max (result, j-i+1);
        j++;
    }
```

```
// second pass
i = 0;
j = 0;
int countT = 0;

while (j < n) {
    if (answerKey.charAt(j) == 'T') {
        countT++;
    }

    while (countT > k) {
        if (answerKey.charAt(i) == 'T') {
            countT--;
        }
        i++;
    }

    result = Math.max(result, j - i + 1);
    j++;
}

return result;
}
```

Leetcode - Max Consecutive Ones III

——One - Pass Sliding Window

Date / /

0	1	2	3	4	5	6	7	8
F	T	F	T	F	T	F	T	F

, K = 2

result = j - i + 1

① i j

F	T	F	T	F	T	F	T	F

F → 2
T → 1
} min ① → K = 2

② i j

F	T	F	T	F	T	F	T	F

F - 2
T - 2
} min ② K = 2

③ i j

F	T	F	T	F	T	F	T	F

F - 3
T - 2
} min (2) K = 2

④ i j

F	T	F	T	F	T	F	T	F

F - 3
T - 3
} 3 (min) > K = 2 ✗

Shrink

⑤ i j

F	T	F	T	F	T	F	T	F

F - 2
T - 3
} min (2) <= K = 2

and so on...

Date __/__/__
Intimate

// Code (One-Pass Solution)

```
public int maxConsecutiveAnswers(String answerKey,
                                 int k) {
    int result = k;

    HashMap<Character, Integer> map =
                    new HashMap<>();
    map.put('T', 0);
    map.put('F', 0);

    int i = 0;

    for (int j = 0; j < answerKey.length(); j++) {
        char ch = answerKey.charAt(j);
        map.put(ch, map.getOrDefault(ch, 0) + 1);

        while (Math.min(map.get('T'),
                map.get('F')) > k) {

            char left = answerKey.charAt(i);
            map.put(left, map.get(left) - 1);
            i++;
        }

        result = Math.max(result, j - i + 1);
    }

    return result;
}
```

Similar - Constrained Subsequence Sum (1425)

Date / /
Intimate

12. Sliding Window Maximum -

Example :- nums = {1, 3, -1, -3, 5, 3, 6, 7}, K = 3

Output :- {3, 3, 5, 5, 6, 7}

// Brute force :-

O(W * K)

W = (n - K + 1)

Date __/__/__
Intimate

⇒ MONOTONIC INCREASING / DECREASING ⇐

1	3	-1	-3	5	3	6	7

```
public int[] maxSlidingWindow(int nums[], int k){
    int n = nums.length;
    int ans[] = new int[n-k+1];

    PriorityQueue<Integer> pq = new PriorityQueue<>
                    ((a,b) -> b-a);

    int i=0;                    GIVING TLE.
    int j=0;

    while(j<n){
        pq.add(nums[j]);

        if(pq.size() == k){
            ans[i] = pq.peek();

            pq.remove(nums[i]);
            i++;
        }
        j++;
    }

    return ans;
}
```

Date / /
Intimate

```
public int[] maxSlidingWindow(int[] nums, int k) {
    ArrayDeque<Integer> q = new ArrayDeque<>();

    int i = 0;
    int j = 0;
    int ptr = 0;
    int n = nums.length;

    int[] res = new int[n-k+1];

    while (j < n) {
        while (!q.isEmpty() && q.peekLast() <
                                nums[j]) {
            q.pollLast();
        }
        q.add(nums[j]);

        if (j-i+1 < k) {
            j++;
        }
        else if (j-i+1 == k) {
            res[ptr] = q.peek();

            if (nums[i] == q.peek()) {
                q.pollFirst();
            }
            i++;
            j++;
        }
    }
    return res;
}
```

Date __/__/__
Intimate

13. Frequency of the Most Frequent

nums = {2, 1, 4} , K = 5

Output = 3

{2, 1, 4}
3 2
4 3
4

K = ~~5~~ 3 ~~2 1~~
0

— Thought Process —

nums = {1, 8, 13, 4} , K = 5

Sorting

① nums = {1, 4, 8, 13} , K = 5 (increment (हो सकता है या क्या) ~)

② for (int i=0; i<n; i++) {

freq = bsearch (i, nums, k);

while (l <= r) {
mid = l + (r-l)/2;
count = (target_idx - mid+1);
windowSum = count * nums [target_idx];

}

Date __/__/____ Intimate

Code (~~Brute force~~) -

1. DRY RUN -

(1) nums = {1, 4, 8, 13}, K = 5
(i=2, j=0)

let's say mid = j

(i - mid + 1) = 2 - 0 + 1 ⇒ 3 (कितने elements हैं बीच में)

window sum ⇒ 3 * 8 = 24 ⇒ 24 - 13 = (11) ops required

old sum ⇒ 1 + 4 + 8 = 13

11 > 5 ✗

(2) nums = {1, 4, 8, 13}, K = 5
(j=1, i=2)

Let's say mid

(i - mid + 1) = 2 - 1 + 1 ⇒ 2

window sum ⇒ 2 * 8 = 16 ⇒ 16 - 12 ⇒ (4) ops.

old sum ⇒ 4 + 8 = 12

4 <= 5 ✓

Date / / Intimate

— STORY Points —

① Sorting

②
```
for (int i = 0; i<n; i++) {
    freq = bsearch (i, nums, k);
    result = max (result, freq);
}
return result;
```

③
```
bsearch (target_idx , k, nums) {
    l = 0;
    r = target_idx;

    while (l <= r) {
        mid = l + (r-l)/2;
        count = target_idx - mid-el;
        windowSize = count * nums[target_idx]
        arrSum = (mid → +) sum;
        ops = windowSum - arr;

        if (ops > k) {
            e = midel
        }
        else {
            result = mid;
            r = mid-1;
        }
    }
```

TC - O(NlogN)
SC - O(N)

Date __/__/__
Intimate

// Code (Binary Search) -

```
public int maxFrequency(int[] nums, int k){
    int n = nums.length;
    Arrays.sort(nums);

    long[] prefixSum = new long[n];
    prefixSum[0] = nums[0];

    for(int i=1; i<n; i++){
        prefixSum[i] = prefixSum[i-1] + nums[i];
    }

    int result = 0;
    for(int targetIdx = 0; targetIdx < n;
            targetIdx++){
        result = Math.max(result,
            bsearch(targetIdx, k, nums, prefixSum));
    }
    return result;
}
```

Date
Instance

```
public int binarySearch(int targetIdx, int k, int[] nums,
    long[] prefixSum) {
  int low = 0, high = targetIdx, ans = -1;
  while (low <= high) {
    int mid = low + (high - low) / 2;
    int len = targetIdx - mid + 1;
    long total = prefixSum[targetIdx] -
        (mid > 0 ? prefixSum[mid - 1] : 0);

    long required = (long) nums[targetIdx] * len;

    if (required - total <= k) {
      ans = len;
      high = mid - 1;
    } else {
      low = mid + 1;
    }
  }
  return ans;
```

Date __/__/__
Intimate

1 => Approach Using Sliding Window —

nums = {1, 8, 13, 4}, K = 5

⇓ After Sorting

① nums = {1, 4, 8, 13}, K = 5
l r

windowSum (ws) = 1 ~~ops <=~~ result = 1
currentSum (cs) = 1 ops = 0 <= k
valid

② nums = {1, 4, 8, 13}, K = 5
l r

ws = 4+4 = 8
cs = 1+4 = 5 ops = 3 <= k result = 2
valid

③ nums = {1, 4, 8, 13}, K = 5
l r

ws = 8+8+8 = 24
cs = 13 , ops > K
Not valid

④ nums = {1, 4, 8, 13}, K = 5
l r

ws = 39
cs = 25 ops > k.
Not valid

⑤ nums = {1, 4, 8, 13}, K = 5
l r
out of bound.

Date __/__/__
Intimate

Code // Sliding Window -

```
public int maxFrequency (int nums [], int k){
    int maxFrequency = 0;
    long currentSum = 0;

    Arrays.sort (nums);
    int left = 0;
    for (int ~~left = 0;~~ right = 0; right < nums.length;
        right ++ ){

        currSum += nums [right];

        while (currentSum + k < (long)
            nums[right] * (right - left + 1 )){
            currentSum -= nums [left];
            left++;
        }

        maxFrequency = Math.max (maxFrequency,
                                 right - left + 1);
    }
    return maxFrequency;
}
```

TC -

SC -

12. Count Complete Substrings — (HARD)

Example:- word = "igigee" , K = 2

Output = 3

[consecutive elements with diff. at max can be only 2]

explain - "igig", "ee", "igigee"?

Some Imp. Observation

1) "aa|bbcc|iii" → Dividing K = 2

(2nd condition tension 2?)

"aa" "bb" "cc" "bbcc" "iii"

1 2 3 4 5

Output ⇒ 5

2) "ddff" → K is given in the qs. (K=2)

dd → unique char = 1 → 1 * K = 2

ff → unique char = 1 → 1 * K = 2

ddff → unique char = 2 → 2 * K = 4

* Find count of substrings in which frequency of all characters = k. Solve function ().

Date ___/___/___ Intimate

"aaa" k = 1
Edge Case

```
public int countCompleteSubstrings(String word, int k){
    int n = word.length();
    int result = 0;
    int last = 0;

    for(int i=1; i<=n; i++){
        if( i == n || Math.abs(word.charAt(i) -
                word.charAt(i-1)) > 2 ){

            result += solve(last, i-1, word, k);
            last = i;
        }
    }

    return result;
}

public int solve(int start, int end, String word,
                 int k){
    int result = 0;
    for(int chars = 1; chars <= 26 &&
        chars * k <= end - start + 1; chars++){
        int count[] = new int[26];
        int goodChars = 0;

        int i = start;
        int windowLength = chars * k;
```

```
for (int j= start; j<= end; j++) {
    char ch = word.charAt(j);

    count[ch-'a']++;

    if (count[ch-'a'] == k) {
        goodChars++;
    }
    else if (count[ch-'a'] == k+1) {
        goodChars--;
    }

    if (j-i+1 > windowLength) {
        if (count[word.charAt(i)-'a'] == k) {
            goodChars--;
        }
        else if (count[word.charAt(i)-'a'] == k+1) {
            goodChars++;
        }
        count[word.charAt(i)-'a']--;
        i++;
    }
    if (goodChars == chars) {
        result++;
    }
  }
 }
return result;
}
```

Date __/__/__
Intimate

Binary Subarrays With Sum –

nums = [1, 0, 1, 0, 1] , goal = 2

↑ ↑ ↑ ↑ ↑

⇒ Using Simple Prefix Sum –

Map

Sum	count
0	1
1	2
2	1

currSum = 1 ⇒ 2+1 ⇒ -1

currSum = 1+0 ⇒ 1-2 ⇒ -1

currSum = 1+0+1 ⇒ 2-2 ⇒ 0

currSum = 1+0+1+0 ⇒ 2-2 ⇒ 0

currSum = 1+0+1+0+1 ⇒ 2-3 ⇒ -1

(absolute val)

result += mp(0) ⇒ 1+1 ⇒ 2 +2 ⇒ 4

Code – (Using Prefix Sum) –

```
public int numSubarraysWithSum(int[] nums, int goal){
    int n = nums.length;
    HashMap<Integer, Integer> mp = new HashMap<>();

    int count = 0;
    int currSum = 0;
    mp.put(0,1);
    for(int num : nums ){
        currSum += num;
        if(mp.containsKey(currSum - goal)){
            count += mp.get(currSum - goal);
        }
        mp.put(currSum, mp.getOrDefault(currSum,0)+1);
    }
    return count;
}
```

Sliding Window —

⇒ {1, 2, 3, 2} , goal = 5

⇒ {1, 0, 1, 0, 1} , goal = 2
i j

windowSum = 1 + 0 + 1 ⇒ 2
windowSum = 1 + 0 + 1 + 0 ⇒ 2
result = 1 + 1

⇒ {1, 0, 1, 0, 1} , goal = 2
i j

windowSum ⇒ 0 + 1 + 0 + 1 ⇒ 2
result = 2

Ex. Dry Run :- {0, 0, 0, 1, 1} , goal = 2
① i j

windowSum = 0 + 0 + 0 + 1 + 1 ⇒ 2 == goal

result ⇒ 1

② {0, 0, 0, 1, 1} , goal = 2
i j

windowSum ⇒ 0 + 0 + 1 + 1 ⇒ goal
result ⇒ 2

③ {0, 0, 0, 1, 1} , goal = 2
i j

windowSum ⇒ 0 + 1 + 1 ⇒ goal
result ⇒ 1

Date __/__/____
Intimate

// Using Sliding Window -

```
public int numSubarraysWithSum(int nums[], int goal){
    int prefixZeros = 0;
    int windowSum = 0;
    int count = 0;

    int i=0, j=0;

    while(j< nums.length){
        windowSum += nums[j];

        while(i<j && (nums[i]==0 ||
                windowSum > goal)){
            if(nums[i] == 1){
                prefixZeros = 0;
            }
            else{
                prefixZeros += 1;
            }
            windowSum -= nums[i];
            i++;
        }
        if(windowSum == goal){
            count += 1 + prefixZeros;
        }
        j++;
    }
    return count;
}
```

Date / /
Intimate

16. Subarray Product Less Than K- {Yatra.com}

Example:- nums = {10, 5, 2, 6} , K = 100

Output - 8

Explanation - {10}, {5}, {2}, {6}, {10, 5}, {5, 2}, {2, 6}, {5, 2, 6}

Similar Problems

→ Maximum size Subarray Sum Equals K
→ Subarray Sum Equals K
→ Subarray having Equal Vowel & consonants
→ Find longest Substring containing vowels in Even Counts
→ Maximum Number of Vowels in a substring of given length.

DRY RUN:-

① nums =

0	1	2	3
10	5	2	6
i, j			

K = 100

product = 10 < K count = 1

{10}

② nums =

0	1	2	3
10	5	2	6
i	j		

product = 10 * 5 = 50 < K

{5}, {10, 5} count ⇒ 1 + (1+1) ⇒ 3

Date
Intimate

3) nums = | 10 | 5 | 2 | 6 | , K = 100
0 1 2 3
i j

product ⇒ 50 * 2 ⇒ 100 ! < 100 count ⇒ 2

Shrink window...

4) nums = | 10 | 5 | 2 | 6 | , K = 100
0 1 2 3
i j

product ⇒ 5 * 2 ⇒ 10 (100/10 ⇒ 10) < 100

{5,2} {2} count ⇒ 3 + 2 ⇒ 5

5) nums = | 10 | 5 | 2 | 6 |
0 1 2 3
i j

product ⇒ 10 * 6 ⇒ 60 < 100 count ⇒ 5 + 3 ⇒ 8

{5,2,6} {2,6} {6}

6) nums = | 10 | 5 | 2 | 6 | j
i

out of bound

Count ⇒ 8

window subarrays ⇒ j - i + 1

TC - ~~O(2N)~~ O(2N) ≈ ~~O(N)~~ O(2N)
SC - O(1)

Date __/__/__
Intimate

// Code - (Sliding window) -

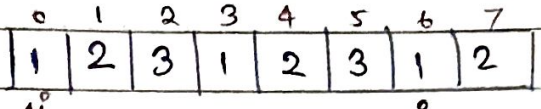
```
public int numSubarrayProductLessThanK(int nums[], int k){
    if (k <= 1) {
        return 0;
    }
    int left = 0;
    int right = 0;
    int product = 1;
    int count = 0;
    int n = nums.length;

    while (right < n) {
        product += nums[right];
        while ( product >= k) {
            product /= nums[left];
            left++;
        }
        count += 1 + (right - left);
        right++;
    }
    return count;
}
```

Date __/__/__
Intimate

17. Length of Longest Subarray With at Most K Frequency -

Example:- nums = {1, 2, 3, 1, 2, 3, 1, 2}, K = 2

Output - 6

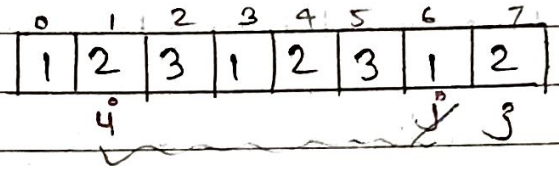


(1)

result = ~~1 2 3 4 5~~ 6

map
1 → ~~1~~ ~~2~~ 3
2 → ~~1~~ 2
3 → ~~2~~ 2

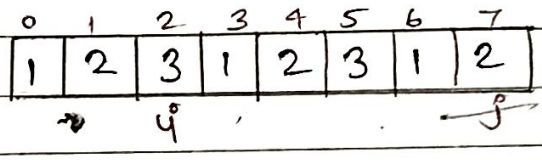
(2)



result = ~~6~~ 6

map
1 → 2
2 → ~~2~~ 3
3 → 2

(3)



j (out of Bound) -

result ⇒ 6

map
1 → 2
2 → 2
3 → 2